```
  for _ in {1..maxSteps} do
                                    #2
    for i in {1.. |P0|} do
                  #3
      P[i] ← P0[i] – alpha * derivative(f, P0,
i)                                #4
    if P == P0
then
    #5
      break
    P0 ←
P
          #6
  return P0
```

#1 Method `gradientDescent` takes a function `f`, a starting point `P0`, a learning rate alpha, and the maximum number of steps to perform, `maxSteps`. It returns a point in the domain: ideally the point where `f` has the smallest value (either locally or globally).

#2 Starts the iteration, running the main cycle at most `maxSteps` times.

#3 Cycle through the coordinates of point `P0`.

#4 Creates a new point `P`, where each coordinate of `P` is assigned the corresponding coordinate of `P0`, with a small delta computed from `f`'s gradient: in particular we need to compute the partial derivative of `f` with respect to its `i`-th coordinate, and then the value it has at point `P0`. The value of the gradient is multiplied by a learning rate `alpha`.

#5 If all the derivatives were 0, we are either in a plateau region, or in a minimum point, and so gradient descent can't improve any further. In reality, we might check that the norm of the difference is not smaller than some precision, both because computer arithmetic has finite precision, and because when the gradient is very small, the possible improvement can be negligible, and not worth the computational resources.